Uriah Raffer
916-213-9145 | umraffer32@gmail.com

Statement of Qualifications

1. What I believe are the responsibilities of a server system specialist

I think the responsibilities start with one question. What is this server actually for? A database server, a web server, and an internal file server do not carry the same risks, so the job shifts with the workload. What stays constant is the goal underneath all of it, which I think of through the CIA triad. Keep the data confidential, keep it intact, and keep it available.

Confidentiality means the server and its data are only reachable by the people and systems that should reach them. That covers authentication, access controls, and locking down how you connect to the box in the first place. Integrity means the data stays accurate and uncorrupted, which is where audit logging and careful change control matter. Availability means the service is there when people need it, and to me that is the part that takes the most engineering. That means redundancy, backups you have actually tested, and a plan for what happens when something fails.

Around that core sit the daily duties. A specialist designs the server and its environment, installs and configures it, and keeps it running through maintenance, updates, and upgrades. You monitor it for problems, troubleshoot to the root cause when something breaks, and keep it patched so known issues do not pile up. You track when hardware and software reach end of life and plan the replacement before it becomes a problem. And you document your work and communicate clearly, because the next person, including future you, has to understand the system.

I built my aws-ha-infra project to put the availability side of this into practice. It is a two-AZ environment on AWS, built with Terraform, that runs the cheaper self-managed NAT instances instead of a managed NAT Gateway and then adds back the reliability the managed option would have handled for you. The part I am most proud of is a self-healing route path. When a NAT instance is replaced and gets a new network interface, an EventBridge rule triggers a Lambda that repairs the private route automatically. It also runs a Multi-AZ PostgreSQL database, SSM-only access with no SSH keys, and CloudWatch and CloudTrail for monitoring and audit logging. I tested it by simulating failures across both availability zones and measuring recovery time. The project is still a work in progress, but building it is how I learned that designing for availability is mostly about planning for the failure you hope never comes.

2. My experience with patch management and system updates

I work with patch management in two settings, a hands-on internship lab and my own homelab.

In the internship, I ran vulnerability and compliance scans on Windows and Linux machines in Microsoft Azure using Tenable. I would run a credentialed scan against a target, pull the findings, and run a compliance audit to see where the machine stood. Then I remediated what I could with PowerShell scripts. On the Windows side that meant Windows 10, Windows 11, and Windows Server. Across several rounds I took a machine from about 30 findings down to 3, eliminating every critical-severity finding along the way.

The three that remained were not loose ends. Two were SSL self-signed certificate findings, expected in a lab that uses internally issued certificates. The third was a high-severity SQLite information-disclosure finding that kept reappearing. It would show on one scan, drop off the next, then come back. Rather than remediate it on reflex, I searched the system with PowerShell for SQLite executables and DLLs and found

none installed. That, plus the finding flapping in and out across scans, told me it was a false positive rather than a real exposure, so I documented it as one instead of forcing a change the machine did not need.

I also learned to verify a fix actually took hold. I remediated an outdated Microsoft Edge install, re-scanned, and it flagged again. That seemed wrong. The update had downloaded, but Edge had to be closed and reopened before it would apply, and I had not done that. Once I closed it, reopened it, and re-scanned, the machine came back clean. Verification ended up cutting both ways. Confirm a real fix landed, and confirm a noisy finding is real before acting on it.

The fixes ranged from removing outdated third-party software like Wireshark and updating software like Edge, to configuration hardening such as enabling SMB signing, requiring Network Level Authentication for RDP, and restoring the LAN Manager authentication level to a secure default. I scripted the repeatable ones in PowerShell so the same fix applied the same way every time.

At home I keep my own systems patched. On my Linux hosts I set up unattended upgrades that run daily, but I usually prefer to patch by hand, so I wrote a short script that runs update, upgrade, autoremove, and autoclean and aliased it to 'up' so I can run it from any directory. I run that across my servers, including two Pi-hole instances and a pfSense firewall I use with Tailscale as an exit node and subnet router. My Windows machines update on their own.

I treat patching as something you keep doing, not a one-time pass. In the lab I also practiced scheduling recurring scans, though that environment spins VMs up and down too often for a standing schedule to fit. The habit I kept is simple. Find out what is out of date or misconfigured, fix it the same way each time, and re-scan to prove it worked.